

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION:

The **Amazon-Inspired E-Commerce Database Management System** is designed to efficiently manage customers, sellers, products, categories, inventory, shopping carts, orders, payments, shipments, and reviews. Functional dependencies help identify the relationship between attributes in each entity, while redundancy analysis and data anomaly detection ensure data consistency and minimize data duplication. A well-designed database improves performance, maintains data integrity, and supports efficient management of online shopping operations.

1. CUSTOMER TABLE:

Functional Dependency

Customer_ID → **First_Name, Last_Name, Email, Phone, Address, City, State, Pincode**

Redundancy Analysis

Customer details are stored only once in the Customer table. Orders, Cart, and Review tables reference the customer using **Customer_ID**, avoiding duplication of customer information.

Data Anomalies

- **Insert Anomaly:** A new customer can register without placing an order.
 - **Update Anomaly:** Updating a customer's phone number or address requires changes only in the Customer table.
 - **Delete Anomaly:** Deleting an order does not remove customer information.
-

2. SELLER TABLE:

Functional Dependency

Seller_ID → **Seller_Name, Email, Phone, Business_Name, Address**

Redundancy Analysis

Seller information is maintained in one table. Products reference sellers using **Seller_ID**, preventing repeated seller details.

Data Anomalies

- **Insert Anomaly:** A seller can register without adding products.
 - **Update Anomaly:** Updating seller details affects only one record.
 - **Delete Anomaly:** Removing a product does not delete seller information.
-

3. CATEGORY TABLE:

Functional Dependency

Category_ID → **Category_Name, Description**

Redundancy Analysis

Each category is stored only once. Products reference the category using **Category_ID**, reducing redundancy.

Data Anomalies

- **Insert Anomaly:** Categories can be added before products.
 - **Update Anomaly:** Category names are updated only once.
 - **Delete Anomaly:** Deleting products does not remove categories.
-

4. PRODUCT TABLE:

Functional Dependency

Product_ID → **Product_Name, Description, Brand, Price, Category_ID**

Redundancy Analysis

Product details are stored only in the Product table. Inventory, Orders, and Reviews reference products using **Product_ID**.

Data Anomalies

- **Insert Anomaly:** New products can be added before customers purchase them.
 - **Update Anomaly:** Product price or description is updated in one place.
 - **Delete Anomaly:** Deleting an order does not remove product information.
-

5. INVENTORY TABLE

Functional Dependency

Inventory_ID → **Product_ID, Stock_Quantity, Last_Updated**

Redundancy Analysis

Inventory details are stored separately from product details. This avoids repeating stock information.

Data Anomalies

- **Insert Anomaly:** Inventory records can be created after adding products.
 - **Update Anomaly:** Stock quantity is updated only in the Inventory table.
 - **Delete Anomaly:** Deleting inventory records does not remove product details.
-

6. CART TABLE:

Functional Dependency

Cart_ID → Customer_ID, Created_Date

Redundancy Analysis

The Cart table stores only cart information. Products inside the cart are maintained through a separate relationship, preventing duplication.

Data Anomalies

- **Insert Anomaly:** A cart can be created before adding products.
 - **Update Anomaly:** Cart details are updated independently.
 - **Delete Anomaly:** Deleting a cart does not remove customer information.
-

7. ORDERS TABLE:

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Orders table stores only order information. Customer details are referenced using **Customer_ID**.

Data Anomalies

- **Insert Anomaly:** Orders can be created for registered customers.
 - **Update Anomaly:** Order status is updated only once.
 - **Delete Anomaly:** Deleting an order does not remove customer information.
-

8. PAYMENT TABLE:

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Amount, Payment_Status, Payment_Date

Redundancy Analysis

Payment details are stored separately from Orders, avoiding duplicate payment records.

Data Anomalies

- **Insert Anomaly:** Payment can be recorded after order creation.
 - **Update Anomaly:** Payment status is updated only once.
 - **Delete Anomaly:** Removing payment information does not delete the order.
-

9. SHIPMENT TABLE:

Functional Dependency

Shipment_ID → Order_ID, Tracking_Number, Delivery_Address, Shipment_Status, Delivery_Date

Redundancy Analysis

Shipment information is stored independently from Orders, preventing repeated delivery details.

Data Anomalies

- **Insert Anomaly:** Shipment details can be added after payment confirmation.
 - **Update Anomaly:** Shipment status is updated only once.
 - **Delete Anomaly:** Deleting shipment information does not remove order details.
-

10. REVIEW TABLE:

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date

Redundancy Analysis

Customer reviews are stored separately from Product and Customer tables. This avoids repeating review information.

Data Anomalies

- **Insert Anomaly:** Reviews can be added only after customers purchase products.
 - **Update Anomaly:** Review comments or ratings are updated only in the Review table.
 - **Delete Anomaly:** Deleting a review does not remove customer or product information.
-

CONCLUSION:

The **Amazon-Inspired E-Commerce Database Management System** is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant information, reducing redundancy and preventing insert, update, and delete anomalies. The database design improves data consistency, maintains integrity, and supports efficient management of customers, products, orders, payments, shipments, inventory, and reviews in an e-commerce environment.